Chapter 26 (Concurrency: An Introduction) — Homework (Simulation)

Zhiyuan Pan

All runs use `x86.py` in `ostep-homework/threads-intro/`. Every command below (and the sweeps over `-i` and `-s`) can be rerun with `code/chapter26/run.sh`.

How to read a trace (`-c`): each row is one instruction. The traced memory location (`-M`) and registers (`-R`) on the left show their values **after** the instruction on the right has run. Each thread has its **own registers** (saved and restored at every switch), but **memory is shared**. A thread runs for `-i` instructions, then an interrupt switches to the next thread. A thread that halts switches right away. With `-r` the interval is random between 1 and `-i`.

Q1

> Let's examine a simple program, "loop.s". First, just read and understand it. Then, run it with these arguments (`./x86.py -t 1 -p loop.s -i 100 -R dx`) This specifies a single thread, an interrupt every 100 instructions, and tracing of register `%dx`. What will `%dx` be during the run? Use the `-c` flag to check your answers; the answers, on the left, show the value of the register (or memory value) after the instruction on the right has run.

```
.top
sub  $1,%dx      # dx = dx - 1
test $0,%dx      # compare dx with 0
jgte .top        # loop while dx >= 0
halt
```

`%dx` starts at 0 (registers default to 0). The first `sub` makes it **-1**. `test` / `jgte` see that -1 < 0, so the loop is not repeated and the thread halts. `%dx` is **-1 for the whole run** after the first instruction:

```
   dx          Thread 0
    0
   -1   1000 sub  $1,%dx
   -1   1001 test $0,%dx
   -1   1002 jgte .top
   -1   1003 halt
```

Only 4 instructions run, so the 100-instruction interrupt never fires.

Q2

> Same code, different flags: (`./x86.py -p loop.s -t 2 -i 100 -a dx=3,dx=3 -R dx`) This specifies two threads, and initializes each `%dx` to 3. What values will `%dx` see? Run with `-c` to check. Does the presence of multiple threads affect your calculations? Is there a race in this code?

Each thread counts `%dx` down: **3 → 2 → 1 → 0 → -1**. The loop runs while `dx >= 0`, so it runs 4 times (dx = 2, 1, 0, -1 after each `sub`). That is 4 × 3 + 1 = 13 instructions per thread. 13 < 100, so thread 0 runs to `halt` before any interrupt, and only then does thread 1 run the same sequence:

```
dx          Thread 0                Thread 1
 3
 2   1000 sub  $1,%dx
...                                                (2, 1, 0, -1)
-1   1003 halt
 3   ----- Halt;Switch -----  ----- Halt;Switch -----
 2                           1000 sub  $1,%dx
...
-1                           1003 halt
```

After the switch the trace shows `dx = 3` again because that column is the **current thread's** `%dx`, and thread 1 has its own copy.

The second thread does **not** change the calculation, and there is **no race**. The code only touches `%dx`, and every thread has its own registers. Nothing is shared between the threads, so no interleaving can change the result.

Q3

> Run this: `./x86.py -p loop.s -t 2 -i 3 -r -R dx -a dx=3,dx=3` This makes the interrupt interval small/random; use different seeds (`-s`) to see different interleavings. Does the interrupt frequency change anything?

With `-i 3 -r` the threads now switch every 1 to 3 instructions, so their instructions interleave. With seed 0:

```
  dx          Thread 0                Thread 1
   3
   2   1000 sub  $1,%dx
   2   1001 test $0,%dx
   2   1002 jgte .top
   3   ------ Interrupt ------  ------ Interrupt ------
   2                           1000 sub  $1,%dx
   ...
   1   1000 sub  $1,%dx
   1   1001 test $0,%dx
   2   ------ Interrupt ------  ------ Interrupt ------
   1                           1000 sub  $1,%dx
   ...
  -1   1003 halt
  ...
  -1                           1003 halt
```

The interleaving is different for seeds 0, 1 and 2, but **the result is always the same**: each thread's `%dx` goes 3, 2, 1, 0, -1, and both threads halt. The interrupt frequency only changes the **order** in which the instructions run, not what they compute. Each thread works only on its own private register, which the OS saves and restores at every switch. Without shared data, there is no race.

Q4

> Now, a different program, `looping-race-nolock.s`, which accesses a shared variable located at address 2000; we'll call this variable `value`. Run it with a single thread to confirm your understanding: `./x86.py -p looping-race-nolock.s -t 1 -M 2000` What is `value` (i.e., at memory address 2000) throughout the run? Use `-c` to check.

```
.top
mov 2000, %ax    # ax = value          \
add $1, %ax      # ax = ax + 1          > critical section
mov %ax, 2000    # value = ax          /
sub  $1, %bx     # bx = bx - 1
test $0, %bx
jgt .top         # loop while bx > 0
halt
```

`%bx` defaults to 0. The loop body runs once: after `sub` bx = -1, which is not greater than 0, so the thread halts. `value` is **0 until the store** at 1002 and **1 from then on**:

```
2000          Thread 0
   0
   0    1000 mov 2000, %ax
   0    1001 add $1, %ax
   1    1002 mov %ax, 2000
   1    1003 sub  $1, %bx
   1    1004 test $0, %bx
   1    1005 jgt .top
   1    1006 halt
```

Q5

> Run with multiple iterations/threads: `./x86.py -p looping-race-nolock.s -t 2 -a bx=3 -M 2000` Why does each thread loop three times? What is final value of `value`?

Each thread loops **three times because `-a bx=3` sets its loop counter** `%bx` to 3. The loop decrements `%bx` and repeats while `bx > 0`, so it runs for bx = 3, 2, 1 and stops when bx reaches 0. `%bx` is a register, so each thread has its own counter.

The final `value` is **6**. Each loop iteration is 6 instructions, so a thread runs 3 × 6 + 1 = 19 instructions, fewer than the default interrupt interval (50). Thread 0 therefore finishes all its updates (value 1, 2, 3) before thread 1 starts, and thread 1 then increments 3 → 6. Nothing interleaves, so the result is correct, but only because of this timing:

```
2000          Thread 0                Thread 1
   0
   1    1002 mov %ax, 2000                                (1st iteration)
   2    1002 mov %ax, 2000                                (2nd)
   3    1002 mov %ax, 2000                                (3rd)
   3    1006 halt
   3    ----- Halt;Switch -----   ----- Halt;Switch -----
   4                             1002 mov %ax, 2000
   5                             1002 mov %ax, 2000
   6                             1002 mov %ax, 2000
   6                             1006 halt
```

Q6

> Run with random interrupt intervals: `./x86.py -p looping-race-nolock.s -t 2 -M 2000 -i 4 -r -s 0` with different seeds (`-s 1`, `-s 2`, etc.) Can you tell by looking at the thread interleaving

> what the final value of `value` will be? Does the timing of the interrupt matter? Where can it safely occur? Where not? In other words, where is the critical section exactly?

Here `%bx` = 0, so each thread runs the loop body once and should add 1. The correct result is 2.

Seed 0 → 2 (correct). Thread 0 runs `mov`, `add`, `mov` (value = 1) and `sub` before the interrupt. Thread 1 then loads 1 and stores 2.

Seed 1 → 1 (wrong, a lost update).

```
2000          Thread 0                Thread 1
   0
   0    1000 mov 2000, %ax                          # T0: ax = 0
   0    ------ Interrupt ------  ------ Interrupt ------
   0                             1000 mov 2000, %ax # T1: ax = 0
   0                             1001 add $1, %ax
   1                             1002 mov %ax, 2000 # T1 stores 1
   1                             1003 sub  $1, %bx
   1    ------ Interrupt ------  ------ Interrupt ------
   1    1001 add $1, %ax                            # T0: ax = 0+1
   1    1002 mov %ax, 2000                          # stores 1: lost
```

Seed 2 → 2. Thread 0 is interrupted only after `sub`, which is after its store.

Yes, the final value can be read off the trace, and the timing of the interrupt is what decides it. I swept seeds 0 to 19 and recorded where thread 0 was first interrupted:

thread 0 interrupted after instruction	seeds	final `value`
1 (`mov 2000,%ax`) or 2 (`add`)	1, 3, 4, 7, 8, 9, 11, 12, 13, 14, 16, 18	**1** (wrong)
3 (`mov %ax,2000`) or later	0, 2, 5, 6, 10, 15, 17, 19	**2** (correct)

The **critical section is exactly the three instructions**

```
mov 2000, %ax
add $1, %ax
mov %ax, 2000
```

An interrupt **after the load and before the store** (after 1000 or after 1001) is unsafe. The thread holds a stale copy of `value` in its private `%ax`, the other thread updates memory, and the store then overwrites that update. An interrupt **anywhere else** (before the load, or after the store, during `sub`, `test`, `jgt`) is safe. Those instructions only use the thread's own `%bx`, and the shared variable in memory is consistent at that point.

Q7

> Now examine fixed interrupt intervals: `./x86.py -p looping-race-nolock.s -a bx=1 -t 2 -M 2000 -i 1` What will the final value of the shared variable `value` be? What about when you change `-i 2`, `-i 3`, etc.? For which interrupt intervals does the program give the "correct" answer?

Each thread adds 1 once, so the correct answer is 2.

- **`-i 1`** → **1.** The threads alternate after every instruction: T0 loads 0, T1 loads 0, T0 adds, T1 adds, T0 stores 1, T1 stores 1. Both read the old value, so one update is lost.
- **`-i 2`** → **1.** T0 runs `mov` + `add` (ax = 1, not stored yet), then T1 runs `mov` + `add` and also loads 0. Both then store 1.
- **`-i 3`** → **2.** T0 runs exactly the three critical-section instructions (value = 1) before the interrupt. T1 then loads 1 and stores 2.

Sweeping `-i 1` to `-i 40`:

`-i`	**1**	**2**	**3**	**4**	**5**	**...**	**40**
final `value`	1	1	**2**	**2**	**2**	**2**	**2**

The answer is correct for **every interval ≥ 3**. With `bx=1` each thread only runs one critical section, at its very start (instructions 1 to 3). Any interval of at least 3 lets thread 0 finish the store before the first interrupt. After that there is no second critical section that could be interrupted.

Q8

> Run the same for more loops (e.g., set `-a bx=100`). What interrupt intervals (`-i`) lead to a correct outcome? Which intervals are surprising?

The correct answer is 200. Sweeping `-i 1` to `-i 40` with `-a bx=100`:

`-i`	value	`-i`	value	`-i`	value	`-i`	value
1	100	11	164	**21**	**200**	31	165
2	100	**12**	**200**	22	164	32	164
3	**200**	13	160	23	168	**33**	**200**
4	150	14	158	**24**	**200**	34	166
5	160	**15**	**200**	25	164	35	169
6	**200**	16	164	26	160	**36**	**200**
7	157	17	166	**27**	**200**	37	161
8	150	**18**	**200**	28	165	38	165
9	**200**	19	163	29	170	**39**	**200**
10	160	20	160	**30**	**200**	40	165

Only intervals that are multiples of 3 give the correct 200. Every other interval loses updates, even very large ones like 37 or 40.

Why multiples of 3. One loop iteration is 6 instructions, and the critical section is the first 3 of them (positions 0, 1, 2 of the iteration). A thread is always interrupted after it has run a multiple of `-i` instructions in its own stream (both threads start at instruction 0 and run equal slices). If `-i` is a multiple of 3, a thread is only ever interrupted at a position that is a multiple of 3 within the iteration. That is after the store (position 3) or at the end of the iteration (position 6), never inside the critical section. For any other interval the interrupt points drift through the loop and sooner or later land between a load and a store.

What is surprising:

- **A larger interval does not mean more correct.** -i 39 is correct but -i 40 loses 35 updates. Correctness depends on how the interval lines up with the loop, not on how rarely interrupts happen. With bx=1 (Q7) every -i ≥ 3 worked, which suggests the wrong conclusion that "long enough slices are safe".
- **-i 1 and -i 2 give exactly 100**: half the updates are lost, the most possible. The threads are in lock step and load the same value every time.
- In real systems the interval is not a fixed instruction count, so there is no "safe" interval. The only fix is to make the three instructions **atomic** with a lock (chapter 28).

Q9

> One last program: `wait-for-me.s`. Run: `./x86.py -p wait-for-me.s -a ax=1,ax=0 -R ax -M 2000` This sets the `%ax` register to 1 for thread 0, and 0 for thread 1, and watches `%ax` and

> memory location 2000. How should the code behave? How is the value at location 2000 being used by the threads? What will its final value be?

```
.main
test $1, %ax     # ax == 1: signaller, ax == 0: waiter
je .signaller
.waiter
mov  2000, %cx   # spin until memory[2000] == 1
test $1, %cx
jne .waiter
halt
.signaller
mov  $1, 2000    # set the flag
halt
```

Thread 0 (ax = 1) is the **signaller**: it writes 1 to address 2000 and halts. Thread 1 (ax = 0) is the **waiter**: it repeatedly reads address 2000 and halts once it sees 1. **Location 2000 is a flag** ("the signaller is done"), used to make the waiter wait for the signaller.

Here the signaller runs first, so the flag is already 1 when the waiter starts. The waiter checks once and halts. The **final value is 1**:

```
 2000      ax          Thread 0                Thread 1
    0       1
    0       1   1000 test $1, %ax
    0       1   1001 je .signaller
    1       1   1006 mov  $1, 2000
    1       1   1007 halt
    1       0   ----- Halt;Switch -----  ----- Halt;Switch -----
    1       0                           1000 test $1, %ax
    1       0                           1001 je .signaller
    1       0                           1002 mov  2000, %cx
    1       0                           1003 test $1, %cx
    1       0                           1004 jne .waiter
    1       0                           1005 halt
```

11 instructions in total.

Q10

> Now switch the inputs: `./x86.py -p wait-for-me.s -a ax=0,ax=1 -R ax -M 2000` How do the threads behave? What is thread 0 doing? How would changing the interrupt interval (e.g., `-i`

> `1000`, or perhaps to use random intervals) change the trace outcome? Is the program efficiently using the CPU?

Now **thread 0 is the waiter**, and it runs first. The flag is still 0, so thread 0 **spins**: it repeats `mov 2000,%cx / test / jne` and makes no progress, until the interrupt at 50 instructions switches to thread 1. Thread 1 sets the flag to 1 and halts. Thread 0 then sees the 1 and halts. The final value is again 1:

```
2000      ax         Thread 0                Thread 1
   0       0   1000 test $1, %ax
   0       0   1001 je .signaller
   0       0   1002 mov  2000, %cx
   0       0   1003 test $1, %cx
   0       0   1004 jne .waiter
               ... the same 3 instructions 16 times in total ...
   0       1   ------ Interrupt ------  ------ Interrupt ------
   0       1                           1000 test $1, %ax
   0       1                           1001 je .signaller
   1       1                           1006 mov  $1, 2000
   1       1                           1007 halt
   1       0   ----- Halt;Switch -----  ----- Halt;Switch -----
   1       0   1002 mov  2000, %cx
   1       0   1003 test $1, %cx
   1       0   1004 jne .waiter
   1       0   1005 halt
```

That is 50 + 4 + 4 = 58 instructions, and 48 of them are thread 0 spinning.

Changing the interrupt interval does not change the outcome (both threads halt, the flag ends at 1), but it changes **how long thread 0 spins**. The waiter cannot finish until the signaller has run, and the signaller only runs after an interrupt. So the waiter burns its entire time slice:

`-i`	instructions executed in total	of which useful (signaller + final check)
50 (default)	58	about 8
100	109	about 9
1000	1009	about 9

With `-i 1000` thread 0 spins for 1000 instructions. With random intervals (`-r`) the length of the spin is random but always lasts until the first interrupt. With Q9's order (signaller first) the interval does not matter at all, because the waiter never has to spin.

The program does not use the CPU efficiently. While thread 0 spins it occupies the CPU doing nothing useful, and it is the very thing preventing the signaller from running. Almost all executed

instructions are wasted, and the waste grows linearly with the time slice. A better design would put the waiter to sleep and have the signaller wake it up (condition variables, chapter 30), or at least have the waiter `yield` the CPU (chapter 28).